Assignment 7 by Chris

On the Wine dataset, I will perform exploratory data analysis (EDA), feature scaling, polynomial feature creation, model training, prediction, and evaluation for a multiclass classification task.

Library Imports

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures, MinMaxScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
```

Load the Wine Dataset & Split into Train-Test Sets

```
# Load the Wine classification dataset as a pandas DataFrame.
wine_data = load_wine(as_frame=True)
df = wine_data.frame

# Split into train-test sets (80/20 split), preserving class proportions.
X_train, X_test, y_train, y_test = train_test_split(
    wine_data.data,
    wine_data.target,
    test_size=0.2,
    random_state=42,
    stratify=wine_data.target
)
```

Exploratory Data Analysis (EDA)

```
print(X_train.info())      # Overview of the dataset
print(X_train.describe())  # Summary statistics
print(X_train.head())      # Display first few rows
```

```
<class 'pandas.DataFrame'>
Index: 142 entries, 36 to 131
Data columns (total 13 columns):
 #   Column                        Non-Null Count  Dtype
---  ------                        --------------  -----
 0   alcohol                       142 non-null    float64
 1   malic_acid                    142 non-null    float64
 2   ash                           142 non-null    float64
 3   alcalinity_of_ash             142 non-null    float64
 4   magnesium                     142 non-null    float64
 5   total_phenols                 142 non-null    float64
 6   flavanoids                    142 non-null    float64
 7   nonflavanoid_phenols          142 non-null    float64
 8   proanthocyanins               142 non-null    float64
 9   color_intensity               142 non-null    float64
 10  hue                           142 non-null    float64
 11  od280/od315_of_diluted_wines  142 non-null    float64
 12  proline                       142 non-null    float64
dtypes: float64(13)
memory usage: 15.5 KB
None
          alcohol  malic_acid         ash  alcalinity_of_ash   magnesium  \
count  142.000000  142.000000  142.000000         142.000000  142.000000
mean    12.971479    2.340000    2.364859          19.625352   99.633803
std      0.802521    1.101285    0.268380           3.380848   14.937448
min     11.030000    0.740000    1.360000          10.600000   70.000000
25%     12.370000    1.610000    2.200000          17.200000   88.000000
50%     13.010000    1.810000    2.360000          19.500000   97.000000
75%     13.560000    3.170000    2.560000          21.500000  107.750000
max     14.830000    5.800000    3.220000          30.000000  162.000000

       total_phenols  flavanoids  nonflavanoid_phenols  proanthocyanins  \
count     142.000000  142.000000            142.000000       142.000000
mean        2.273732    1.985352              0.360141         1.600211
std         0.621454    0.951952              0.119211         0.579777
```

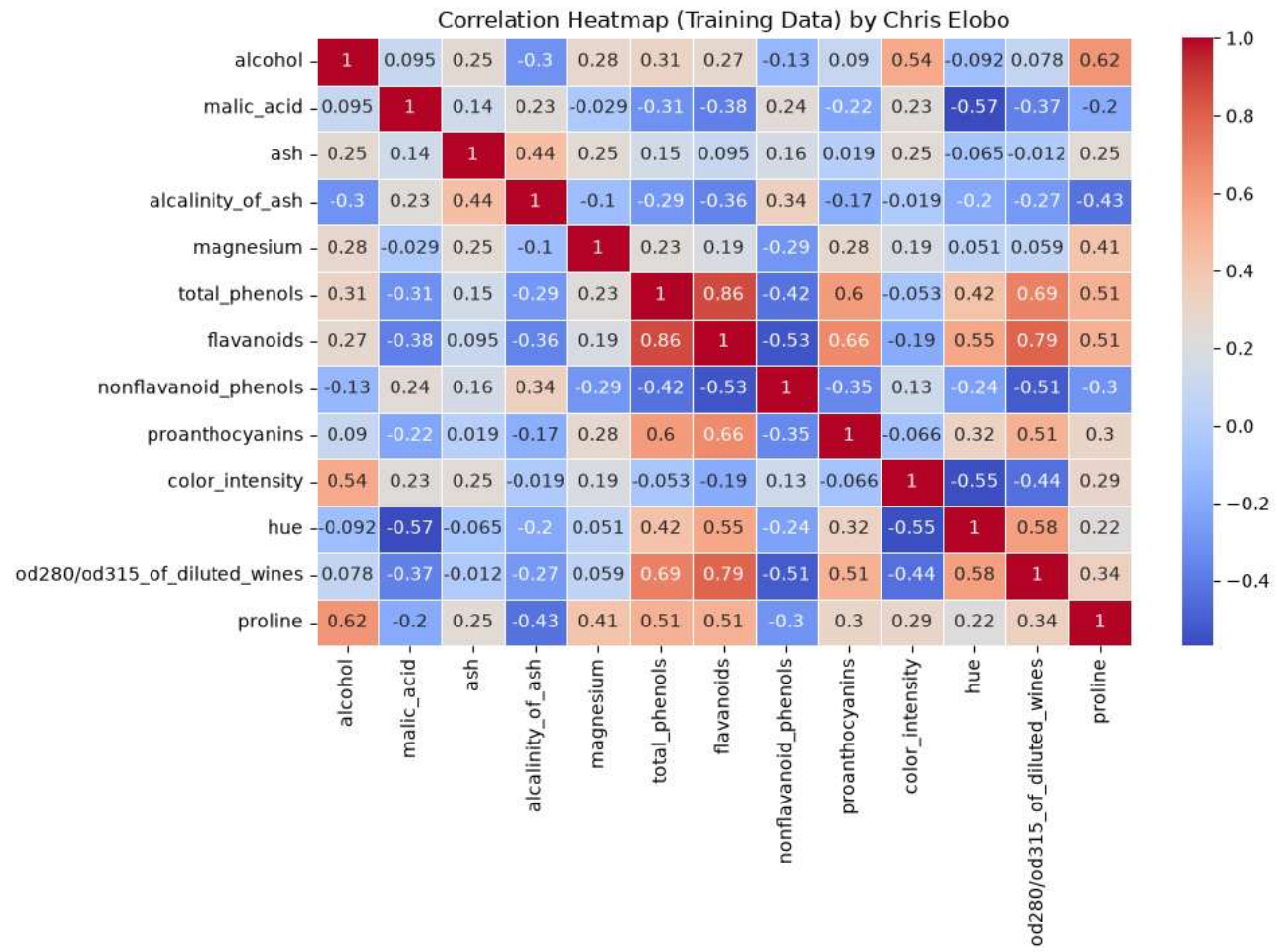
```
min         0.980000    0.340000              0.130000         0.420000
25%         1.705000    1.227500              0.270000         1.250000
50%         2.335000    2.135000              0.340000         1.560000
75%         2.800000    2.775000              0.430000         1.950000
max         3.880000    3.740000              0.630000         3.580000

       color_intensity         hue  od280/od315_of_diluted_wines      proline
count       142.000000  142.000000                    142.000000   142.000000
mean          4.989648    0.948986                      2.606197   739.478873
std           2.334947    0.226647                      0.689650   301.499014
min           1.280000    0.480000                      1.270000   278.000000
25%           3.057500    0.780000                      1.970000   496.250000
50%           4.640000    0.960000                      2.780000   673.500000
75%           6.182500    1.090000                      3.160000   981.250000
max          13.000000    1.710000                      3.920000  1515.000000
     alcohol  malic_acid   ash  alcalinity_of_ash  magnesium  total_phenols  \
36     13.28        1.64  2.84               15.5      110.0           2.60
30     13.73        1.50  2.70               22.5      101.0           3.00
26     13.39        1.77  2.62               16.1       93.0           2.85
12     13.75        1.73  2.41               16.0       89.0           2.60
148    13.32        3.24  2.38               21.5       92.0           1.93

     flavanoids  nonflavanoid_phenols  proanthocyanins  color_intensity   hue  \
```

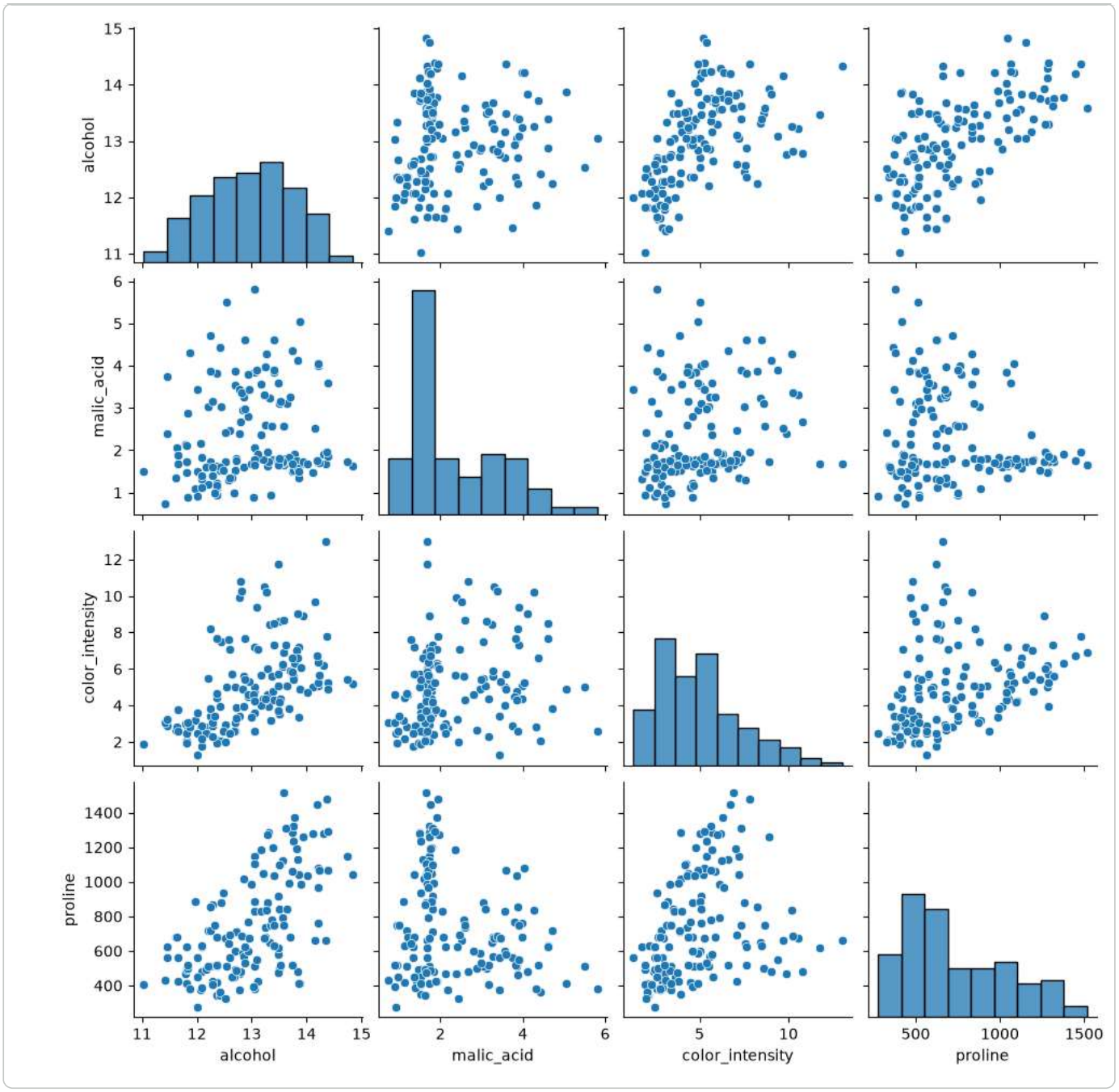
Correlation Heatmap

```
plt.figure(figsize=(10, 6))
sns.heatmap(X_train.corr(), annot=True, cmap="coolwarm", linewidths=0.5)
plt.title("Correlation Heatmap (Training Data) by Chris Elobo")
plt.show()
```



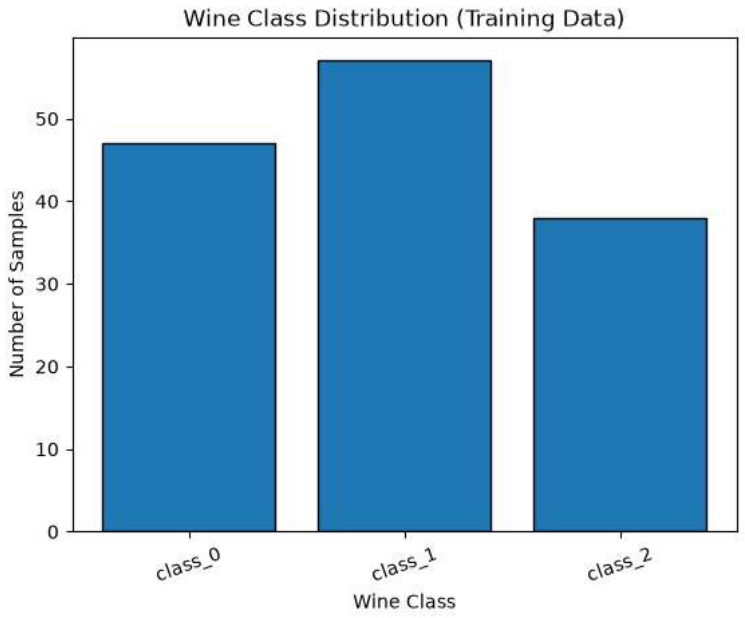
Pairplot of Selected Features

```
sns.pairplot(X_train[['alcohol', 'malic_acid', 'color_intensity', 'proline']])
plt.show()
```



Wine Class Distribution (Training Set)

```
class_counts = y_train.value_counts().sort_index()
plt.bar(wine_data.target_names[class_counts.index.to_numpy()], class_counts.values, edgecolor='black')
plt.title("Wine Class Distribution (Training Data)")
plt.xlabel("Wine Class")
plt.ylabel("Number of Samples")
plt.xticks(rotation=20)
plt.show()
```



Feature Scaling (StandardScaler)

```
scaler = StandardScaler()

# Scale the training set
X_train_scaled = scaler.fit_transform(X_train)

# Scale the test set
X_test_scaled = scaler.transform(X_test)

# Convert scaled data back into DataFrames for easier handling
X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=X_train.columns)
X_test_scaled_df = pd.DataFrame(X_test_scaled, columns=X_test.columns)

# Inspect the scaled training set
print(X_train_scaled_df.head())
```

```
    alcohol  malic_acid       ash  alcalinity_of_ash  magnesium  \
0  0.385801   -0.637871  1.776668          -1.224532   0.696430
1  0.948519   -0.765445  1.253174           0.853284   0.091785
2  0.523354   -0.519409  0.954034          -1.046433  -0.445678
3  0.973529   -0.555859  0.168793          -1.076116  -0.714409
4  0.435820    0.820120  0.056615           0.556453  -0.512860

   total_phenols  flavanoids  nonflavanoid_phenols  proanthocyanins  \
0       0.526865    0.732292             -0.169549        -0.415783
1       1.172795    1.333181             -0.590457         1.349742
2       0.930572    1.006382             -0.169549        -0.260002
3       0.526865    0.816627             -0.590457         0.363125
4      -0.555068   -1.291756              0.756449        -0.606183

   color_intensity       hue  od280/od315_of_diluted_wines   proline
0        -0.167467  0.624378                      0.252908  0.467725
1         0.305303  1.067155                      0.151048  1.815768
2        -0.081509 -0.128343                      0.893172  1.516203
3         0.262324  0.890044                      0.427526  1.932265
4         1.474335 -1.766619                     -1.435059 -0.297831
```

Create Polynomial Features

```
poly = PolynomialFeatures(degree=2, include_bias=False)

# Create polynomial features for training and test sets
X_train_poly = poly.fit_transform(X_train_scaled_df)
X_test_poly = poly.transform(X_test_scaled_df)
```

Model Training (Multiclass Logistic Regression)

```
lr_model = LogisticRegression(max_iter=5000)
lr_model.fit(X_train_poly, y_train)
```

```
▾ LogisticRegression ⓘ ?
▸ Parameters
▸ Fitted attributes
```

Model Prediction and Evaluation

```
y_pred = lr_model.predict(X_test_poly)
print(f"Test accuracy: {accuracy_score(y_test, y_pred):.3f}")
print(classification_report(y_test, y_pred, target_names=wine_data.target_names))
```

```
Test accuracy: 0.972
              precision    recall  f1-score   support

     class_0       1.00      1.00      1.00        12
     class_1       0.93      1.00      0.97        14
     class_2       1.00      0.90      0.95        10

    accuracy                           0.97        36
   macro avg       0.98      0.97      0.97        36
weighted avg       0.97      0.97      0.97        36
```

Feature Scaling (MinMaxScaler)

```
min_max_scaler = MinMaxScaler()

# Min-Max scale the training and test sets
X_train_minmax = min_max_scaler.fit_transform(X_train)
X_test_minmax = min_max_scaler.transform(X_test)

# Convert to DataFrames for easier handling
X_train_minmax_df = pd.DataFrame(X_train_minmax, columns=X_train.columns)
X_test_minmax_df = pd.DataFrame(X_test_minmax, columns=X_test.columns)

# Inspect the scaled training set
print(X_train_minmax_df.head())
```

```
    alcohol  malic_acid       ash  alcalinity_of_ash  magnesium  \
0  0.592105    0.177866  0.795699           0.252577   0.434783
1  0.710526    0.150198  0.720430           0.613402   0.336957
2  0.621053    0.203557  0.677419           0.283505   0.250000
3  0.715789    0.195652  0.564516           0.278351   0.206522
4  0.602632    0.494071  0.548387           0.561856   0.239130

   total_phenols  flavanoids  nonflavanoid_phenols  proanthocyanins  \
0       0.558621    0.688235                  0.42         0.297468
1       0.696552    0.855882                  0.32         0.620253
2       0.644828    0.764706                  0.42         0.325949
3       0.558621    0.711765                  0.32         0.439873
4       0.327586    0.123529                  0.64         0.262658

   color_intensity       hue  od280/od315_of_diluted_wines   proline
0         0.283276  0.495935                      0.569811  0.486661
1         0.377133  0.577236                      0.543396  0.814066
2         0.300341  0.357724                      0.735849  0.741310
3         0.368601  0.544715                      0.615094  0.842361
4         0.609215  0.056911                      0.132075  0.300728
```

Further Analysis: L1-Regularized Classification

```
from sklearn.linear_model import LogisticRegression

# Train a Logistic Regression model with L1 regularization.
lasso_model = LogisticRegression(solver='saga', l1_ratio=1.0, C=1.0, max_iter=5000, random_state=42)
lasso_model.fit(X_train_poly, y_train)

# Predict on the test set
y_pred_lasso = lasso_model.predict(X_test_poly)
```

```
# Evaluate the L1-regularized classifier.
print(f"L1 Logistic Regression accuracy: {accuracy_score(y_test, y_pred_lasso):.3f}")
```

```
L1 Logistic Regression accuracy: 0.972
```

Cross-Validation (Classification Accuracy):

```
from sklearn.model_selection import cross_val_score

# Evaluate multiclass classification with stratified cross-validation.
cross_val_accuracy = cross_val_score(lr_model, X_train_poly, y_train, cv=5, scoring='accuracy')
print(f"Cross-Validated Accuracy: {cross_val_accuracy.mean():.3f} +/- {cross_val_accuracy.std():.3f}")
```

```
Cross-Validated Accuracy: 0.986 +/- 0.017
```